CPU Lite — Design and Implementation Report

Micro-architecture • RTL • Lint/CDC • Constraints • Synthesis • STA

Tirupati Nikhilesh - 6944

1. Project overview

CPU Lite is a pipelined, 32-bit datapath RISC CPU with a 12-bit word-addressed program counter, 16 general-purpose registers (R0–R15), and a 4-bit program status register (PSR: Z, N, C, V). Program memory is read-only and sized 4K × 32 words. The data memory is 16K × 32 words and is accessed through an L1 data cache.

The CPU core and L1 cache operate in the fast clock domain (nominally 150 MHz); the data-memory-side logic operates in a slower, asynchronous clock domain (nominally 70 MHz). Requests and responses cross the domain boundary through asynchronous FIFOs in the CDC bridge. The supplied block diagrams show CPU_Core, CPU_Top, and Memory System boundaries.

2. Micro-architecture specification

2.1 Top-level block diagram

The supplied CPU_Top diagram shows three principal blocks: Program Memory, CPU Core, and Memory System. CPU Core outputs request controls/address/write data and receives read data/ready from Memory System. CPU_Top distributes clk and rst_n to the core and program memory, and mem_clk to the memory system.

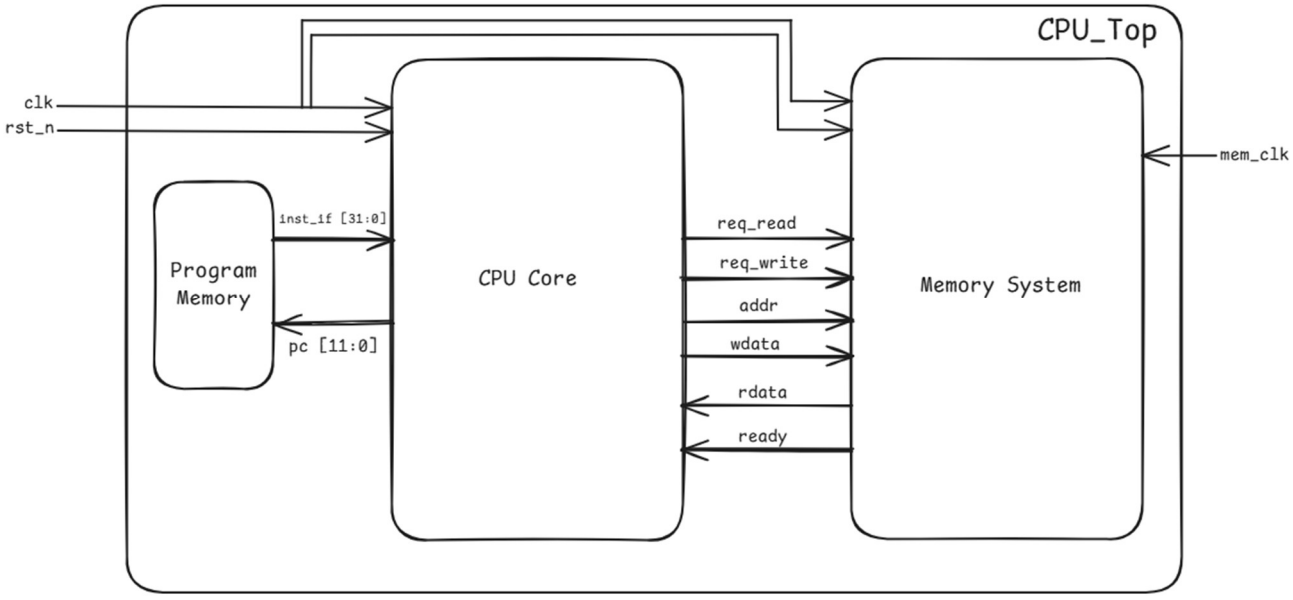
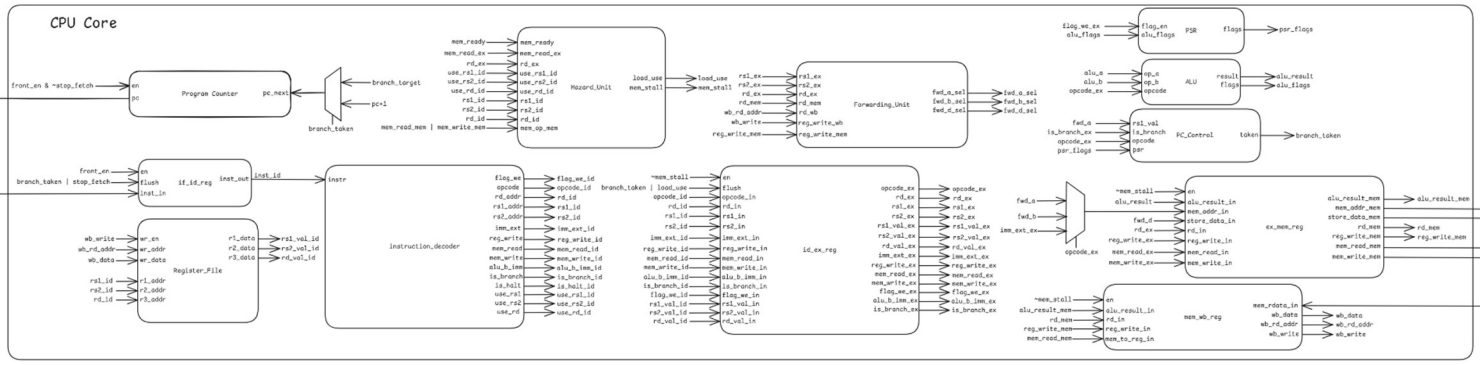


Figure 1. CPU_Top integration diagram supplied with the request.

2.2 CPU core sub-blocks



Sub-block	Responsibility / interface summary	Key widths
Program Counter (PC)	Maintains instruction word address; selects sequential PC+1 or branch target; front-end enable/stop/flush controls update behavior.	PC/address: 12 bits; PC+1 result: 12 bits
IF/ID register	Holds fetched instruction and associated fetch/decode state; supports flush and stall/enable.	Instruction: 32 bits;
Instruction decoder	Decodes opcode and fields, generates register indices, immediate, ALU operation and memory/branch controls.	Instruction: 32 bits; register index: 4 bits; PSR: 4 bits.
Register file	Sixteen general-purpose registers; two read ports and one write port as shown.	16 × 32 bits; 4-bit addresses; read/write data 32 bits.
Hazard unit	Detects load-use and other dependency conditions; drives pipeline stall/flush and source-use control.	Register address 4 bits; controls 1 bit each
ID/EX pipeline register	Carries decoded operands, register indices, immediate and control signals into execute stage; supports stall/flush.	Operands/immediate: 32 bits;
Forwarding unit	Selects forwarded operands from later pipeline stages to resolve data hazards.	Forwarding selects shown as control signals;
ALU	Performs arithmetic/logic operations and produces result and flags.	Inputs/result: 32 bits; opcode width 8; flags 4 bits.
PC control	Evaluates branch condition using forwarded operand, opcode and PSR flags; produces branch-taken.	Condition inputs: 32-bit value and 4-bit PSR; branch control 1 bit.
PSR	Stores condition flags Z/N/C/V when enabled.	4 bits; exact update/priority logic set according to OPCODE.
EX/MEM pipeline register	Carries ALU result, memory address/data, destination register and memory/writeback controls.	Data/address: 32 bits; register address: 4 bits.
MEM/WB pipeline register	Selects and carries memory or ALU result and writeback controls to register file.	Writeback data: 32 bits; register address: 4 bits.

2.3 Memory-system block diagram

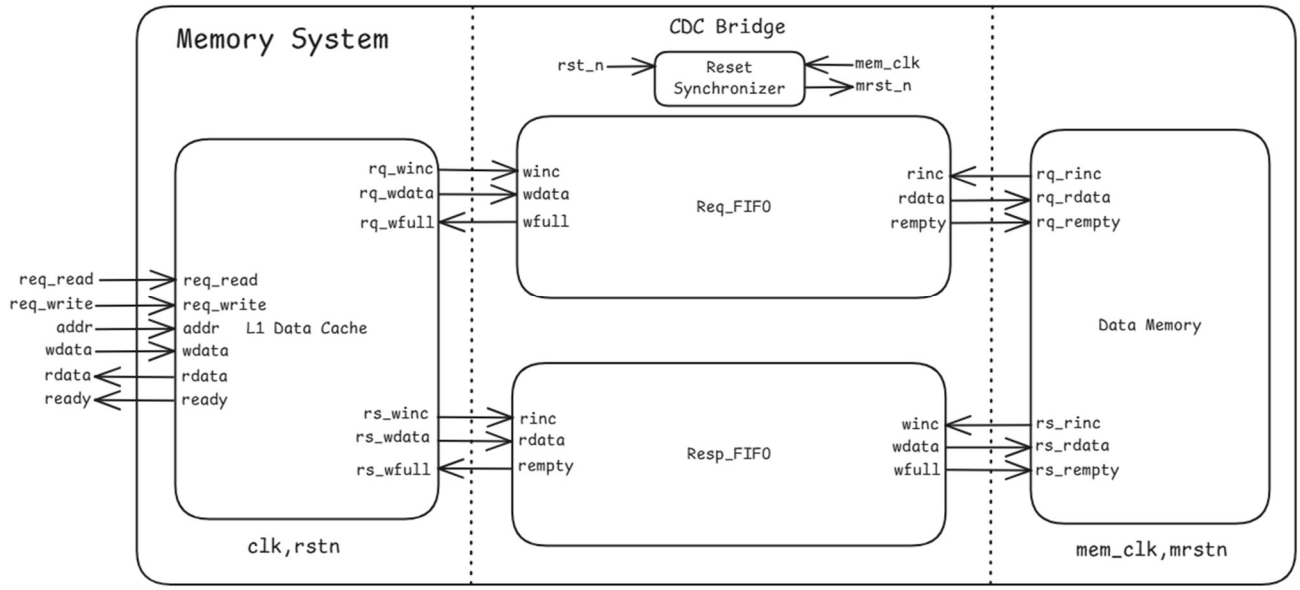
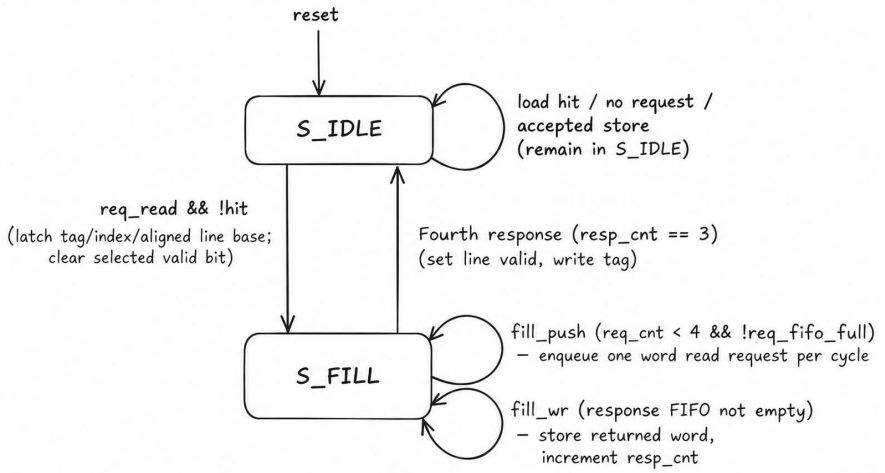


Figure 2. Memory System / CDC bridge diagram supplied with the request.

2.4 Memory-system sub-blocks

Block	Description	Widths / capacity
L1 data cache	First point of access for LOAD/STORE. Cache hits complete locally; misses issue requests through the request FIFO and return refill/response data through the response FIFO.	14-bit word address; 256 lines × 4 words = 1024 words; tag 4 bits, index 8 bits, word offset 2 bits; 32-bit words.
Request asynchronous FIFO	Transfers cache miss requests from fast CPU/cache domain to slow memory domain. Diagram shows write-side signals rq_winc/rq_wdata/rq_wfull and read-side rq_rinc/rq_rdata/rq_rempty.	47-bit payload; PTR_W=3 gives 8 entries.
Response asynchronous FIFO	Transfers memory responses from slow memory domain back to fast cache domain. Diagram shows rs_winc/rs_wdata/rs_wfull and rs_rinc/rs_rdata/rs_rempty.	32-bit payload; PTR_W=3 gives 8 entries.
Data memory	Word-addressed 16K × 32 memory on mem_clk domain; accepts requests and returns read data/status.	Address: 14 bits for 16K words; data: 32 bits.
Reset synchronizer	Synchronizes reset release into memory clock domain, generating mrst_n from rst_n and mem_clk as shown.	Two-flop synchronizer.

2.5 FSM state diagrams



Cache FSM:
S_IDLE -- req_read && !hit --> S_FILL (latch tag/index/aligned line base; clear selected valid bit)
S_IDLE -- load hit / no request / accepted store --> remain S_IDLE
S_FILL -- fill_push when req_cnt < 4 and request FIFO not full; enqueue one word read request per cycle
S_FILL -- fill_wr when response FIFO not empty; store returned word and increment resp_cnt
S_FILL -- fourth response (resp_cnt == 3) --> set line valid, write tag, return to S_IDLE

2.6 Datapath widths and memory map

Item	Width / capacity	Notes
Instruction	32 bits	Program memory word width.
PC / program address	12 bits	4K word-addressed program memory; PC increments by one word.
General-purpose registers	16 × 32 bits	R0–R15; register index is 4 bits.
PSR	4 bits	Z, N, C, V; supplied context states PSR[2] is not required for any operation, but exact mapping should be documented from RTL.
Data address	14 bits	16K × 32 word-addressed data memory.
Data word	32 bits	One address selects one 32-bit word.
Cache line	4 words = 128 data bits; 256 lines; 1024 words total	14-bit word address: tag[13:10], index[9:2], word offset[1:0].
Request FIFO payload	47 bits	{write_enable[46], word_address[45:32], write_data[31:0]}.
Response FIFO payload	32 bits	Read data only; no response status field.

2.7 Clock and reset architecture

Domain	Clock	Nominal frequency / period	Reset
CPU core, pipeline and L1 cache	clk	150 MHz / 6.66 ns	rst_n (active-low)
Data memory and memory-side FIFO logic	mem_clk	70 MHz / 14.28 ns	mrst_n generated/synchronized from rst_n;
CDC boundary	Asynchronous	clk and mem_clk are unrelated clocks	Request and response async FIFOs;

2.8 CDC inventory table

Crossing ID	Source → destination	Class	Synchronizing structure
CDC-01	CPU/cache clk → memory-side mem_clk: request payload	Multi-bit bundled data crossing	Request asynchronous FIFO (rq_winc/rq_wdata/wfull; read side rq_rinc/rq_rdata/rempty)
CDC-02	Memory-side mem_clk → CPU/cache clk: response payload	Multi-bit bundled data crossing	Response asynchronous FIFO (rs_winc/rs_wdata/wfull; read side rs_rinc/rs_rdata/rempty)
CDC-03	rst_n → mem_clk reset domain	Reset-domain crossing	Reset synchronizer producing mrst_n

3. Register map

Name	Width	Access	Reset value	Bit-field description
R0	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R1	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R2	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R3	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R4	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R5	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R6	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R7	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R8	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers.

				Not memory-mapped.
R9	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R10	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R11	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R12	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R13	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R14	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
R15	32	RW (internal)	0x00000000	General-purpose operand/destination register; 4-bit register index; reset loop clears all 16 registers. Not memory-mapped.
PC	12	Internal state (not software-readable via an instruction)	0x000	Word address of instruction; increments by 1 or loads branch_target; asynchronously reset to zero.
PSR	4	Internal state (flag updates controlled per bit)	0x0	PSR[0]=Z, [1]=N, [2]=C, [3]=V; each flag updates only when corresponding flag_en bit is asserted; asynchronously reset to zero.

The instruction encoding is opcode[31:24], rd[23:20], rs1[19:16], rs2[15:12], imm[11:0]. Immediate extension is zero-extension by default and sign-extension for ADDI/SUBI.

4. RTL source package

4.1 Source file inventory

File	Purpose
cpu_top.v	Top-level integration; instantiates cpu_core, program_memory and memory_system; INIT_FILE defaults to prog.mem.
cpu_core.v	Five-stage IF/ID/EX/MEM/WB pipeline integration; hazard handling, forwarding, branch control, PSR and halt state.
program_memory.v	Read-only 4096 × 32 instruction memory; zero-initialized then loaded with $readmemh; module default INIT_FILE is prog.hex.
memory_system.v	Integrates reset_sync, 47-bit request FIFO, 32-bit response FIFO, L1 cache and data_memory.
l1_data_cache.v	256 direct-mapped lines × 4 words/line = 1024 words; 14-bit word address split into tag[13:10], index[9:2], offset[1:0]; write-through store requests and four-word fill on load miss.
async_fifo.v	Parameterized dual-clock asynchronous FIFO; DATA_W parameter, PTR_W=3 default gives depth 8; binary pointers address storage and Gray-coded pointers cross domains.
data_memory.v	16,384 × 32-bit word-addressed data memory; accepts 47-bit requests and emits 32-bit read responses.
reset_sync.v	Two-flop reset synchronizer; asynchronous assertion and synchronous deassertion in destination clock domain.
alu.v	32-bit combinational ALU with 8-bit opcode and 4-bit flags.
instruction_decoder.v	Decodes 32-bit instructions; opcode[31:24], rd[23:20], rs1[19:16], rs2[15:12], immediate[11:0].
register_file.v	16 × 32-bit register file, two source ports plus third rd/store-data read port and one write port; reset clears all registers.
program_counter.v	12-bit PC register with enable and asynchronous active-low reset.
if_id_reg.v	Instruction pipeline register; parameterized instruction width (default 32), supports enable and flush.
id_ex_reg.v	Carries decoded operands and control into EX; includes 32-bit operand/immediate fields and 4-bit flag write enable.
ex_mem_reg.v	Carries ALU result, memory address, store data, destination and memory/writeback controls into MEM.
mem_wb_reg.v	Selects memory read data or ALU result for 32-bit writeback; carries 4-bit destination and write enable.
hazard_unit.v	Detects load-use hazards and stalls while a memory operation is not ready.
forwarding_unit.v	Selects register-file, EX/MEM or MEM/WB values for rs1, rs2 and store-data forwarding.
pc_control.v	Evaluates JMP/JMP_IF and conditional branches using PSR flags.
psr.v	Four-bit Z/N/C/V flag register with per-bit enables and asynchronous active-low reset.

5. Lint report and waiver file

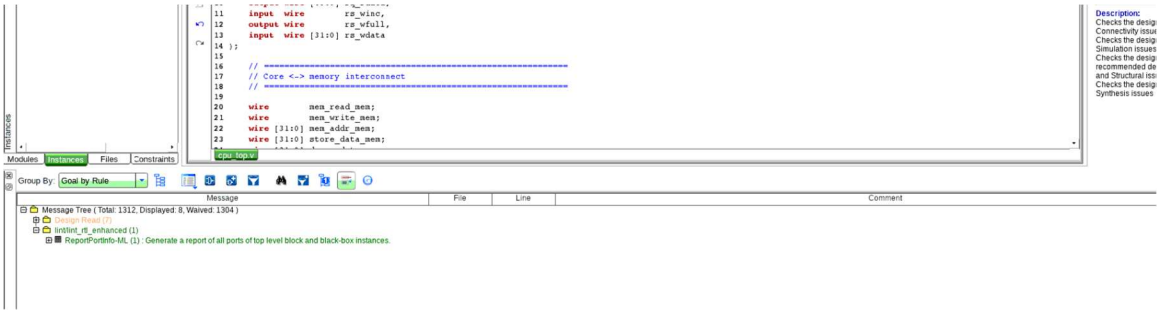


Figure 3. SpyGlass lint results screenshot (lint/lint_rtl_enhanced).

5.1 Waived Rules

Rule	Code-based justification
STARC05-2.11.3.1	The reported construct is an intentional coding-style choice in the reviewed RTL and is retained because its behavior matches the intended design; verify the exact flagged line before sign-off.
STARC-2.3.4.3	The cache data and tag arrays are not reset because reset clears every valid bit, making all lines invalid until refilled; this is safe only while all hits are gated by the valid bit.
W164a	The flagged width truncation is intentional only where the destination width is explicitly narrower by design; confirm the exact source/destination widths and that discarded upper bits are not required.
W240	PSR bit 2 is the carry flag and is updated by the ALU, but current branch decoding uses Z, N and V rather than C; waive only if the rule concerns its lack of consumption by current instructions.
Reset_sync04	The two-flop reset synchronizer intentionally deasserts reset synchronously to mem_clk while asserting asynchronously, as implemented by reset_sync; confirm both flops remain in the destination domain.
Ac_conv01	The flagged conversion is intentional for the specific signal-width or representation conversion in the RTL; confirm the exact conversion preserves the intended value and does not silently lose required bits.
Ac_conv02	The flagged conversion is associated with the intended FIFO/cache data-path behavior; waived only after confirming the converted signal widths and FIFO payload packing match the 47-bit request and 32-bit response interfaces.

6. CDC report and inventory

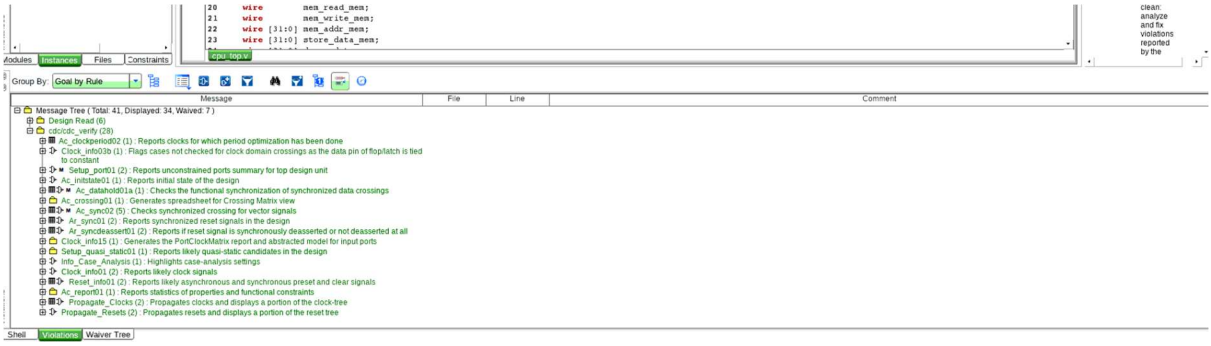


Figure 4. SpyGlass CDC results screenshot (cdc/cdc_verify).

6.1 FIFO SGDC

```
current_design async_fifo

clock -name wclk \
      -domain WCLK \
      -period 6.66 \
      -edge {0 3.33}

clock -name rclk \
      -domain RCLK \
      -period 14.286 \
      -edge {0 7.143}

reset -name wrst_n -value 0
reset -name rrst_n -value 0

fifo -memory mem
```

7. SDC constraints

Sl. No.	Constraint	SDC Command	Justification
1	CPU Clock	create_clock -name clk -period 6.66 [get_ports clk]	Defines the 150 MHz CPU clock.
2	Memory Clock	create_clock -name mem_clk -period 14.28 [get_ports mem_clk]	Defines the 70 MHz memory clock.
3	Asynchronous Clock Groups	set_clock_groups -asynchronous -group [get_clocks clk] -group [get_clocks mem_clk]	Declares CPU and memory clocks asynchronous because they operate in independent clock domains.
4	CPU Setup Uncertainty	set_clock_uncertainty -setup 0.66 [get_clocks clk]	Accounts for setup clock uncertainty in the CPU domain.
5	Memory Setup Uncertainty	set_clock_uncertainty -setup 1.42 [get_clocks mem_clk]	Accounts for setup clock uncertainty in the memory domain.
6	CPU Input Maximum Delay	set_input_delay -clock clk -max 4.00 [get_ports {inst_if}]	Specifies maximum external input delay for the instruction input.
7	CPU Input Minimum Delay	set_input_delay -clock clk -min 0.20 [get_ports {inst_if}]	Specifies minimum external input delay for hold analysis.
8	CPU Output Maximum	set_output_delay -clock clk -max 4.00 [get_ports	Specifies maximum external output delay for the

	Delay	{pc}]	program counter.
9	CPU Output Minimum Delay	set_output_delay -clock clk -min 0.20 [get_ports {pc}]	Specifies minimum external output delay for hold analysis.
10	Memory Input Maximum Delay	set_input_delay -clock mem_clk -max 8.50 [get_ports {rq_rinc rs_winc rs_wdata}]	Specifies maximum external input delay in the memory domain.
11	Memory Input Minimum Delay	set_input_delay -clock mem_clk -min 0.50 [get_ports {rq_rinc rs_winc rs_wdata}]	Specifies minimum external input delay for hold analysis.
12	Memory Output Maximum Delay	set_output_delay -clock mem_clk -max 8.50 [get_ports {rq_rdata rq_rempty rs_wfull}]	Specifies maximum external output delay in the memory domain.
13	Memory Output Minimum Delay	set_output_delay -clock mem_clk -min 0.50 [get_ports {rq_rdata rq_rempty rs_wfull}]	Specifies minimum external output delay for hold analysis.

8. Synthesis reports

8.1 Area report

Metric	Reported value
Design	cpu_top
Library	saed32lvt_ss0p75v25c
Ports	590
Nets	105,387
Cells	104,188
Combinational cells	68,581
Sequential cells	35,603
Macros / black boxes	0
Buffers / inverters	3,331
Combinational area	174,891.989406
Buffer/inverter area	6,166.804159
Non-combinational area	235,871.562298
Macro/black-box area	0.000000
Net interconnect area	504,332.828268
Total cell area	410,763.551704
Total area	915,096.379972

8.2 Timing/QoR summary

Clock group	Path type	Startpoint → endpoint	Slack
clk	Setup / max	u_core/u_ex_mem/rd_mem_reg[2] → u_core/u_psr/flags_reg[0]	0.37 ns
mem_clk	Setup / max	rs_winc input → u_mem/u_resp_fifo/wfull_reg	2.92 ns
clk	Hold / min	u_core/u_id_ex/flag_we_ex_reg[3] → u_core/u_psr/flags_reg[3]	0.22 ns
mem_clk	Hold / min	u_mem/u_req_fifo/wptr_s1_reg[0] → u_mem/u_req_fifo/wptr_s2_reg[0]	0.22 ns

9. STA reports

9.1 Setup summary

Fast clock: worst supplied max-delay path starts at u_core/u_ex_mem/rd_mem_reg[2] and ends at u_core/u_psr/flags_reg[0]. Data arrival is 5.55 ns; required time is 5.92 ns after 0.66 ns setup uncertainty and 0.08 ns library setup time; reported slack is +0.37 ns.

Memory clock: worst supplied max-delay path starts at input rs_winc with 8.50 ns external delay and ends at u_mem/u_resp_fifo/wfull_reg. Data arrival is 9.86 ns; required time is 12.77 ns after 1.42 ns setup uncertainty and 0.09 ns library setup time; reported slack is +2.92 ns.

9.2 Hold summary

Fast clock: the supplied minimum-delay path from u_core/u_id_ex/flag_we_ex_reg[3] to u_core/u_psr/flags_reg[3] reports +0.22 ns hold slack.

Memory clock: the supplied minimum-delay path from u_mem/u_req_fifo/wptr_s1_reg[0] to u_mem/u_req_fifo/wptr_s2_reg[0] reports +0.22 ns hold slack.

10. Simulation Waveforms

The following waveform show CPU pipeline simulation behavior, including clock/reset, program-counter progression, fetched and decoded instructions, operand values, ALU results, memory-stage signals, stall/ready behavior, and writeback activity.

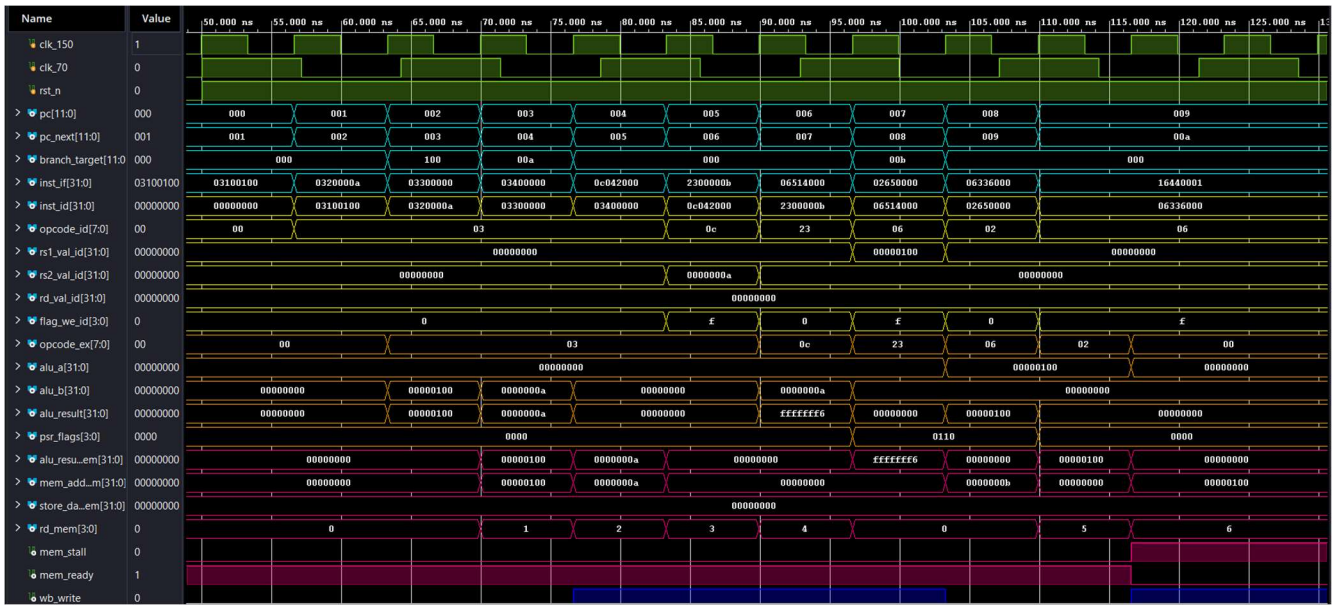


Figure 5. CPU pipeline waveform showing clock/reset, PC progression, instruction flow, ALU and memory-stage signals.

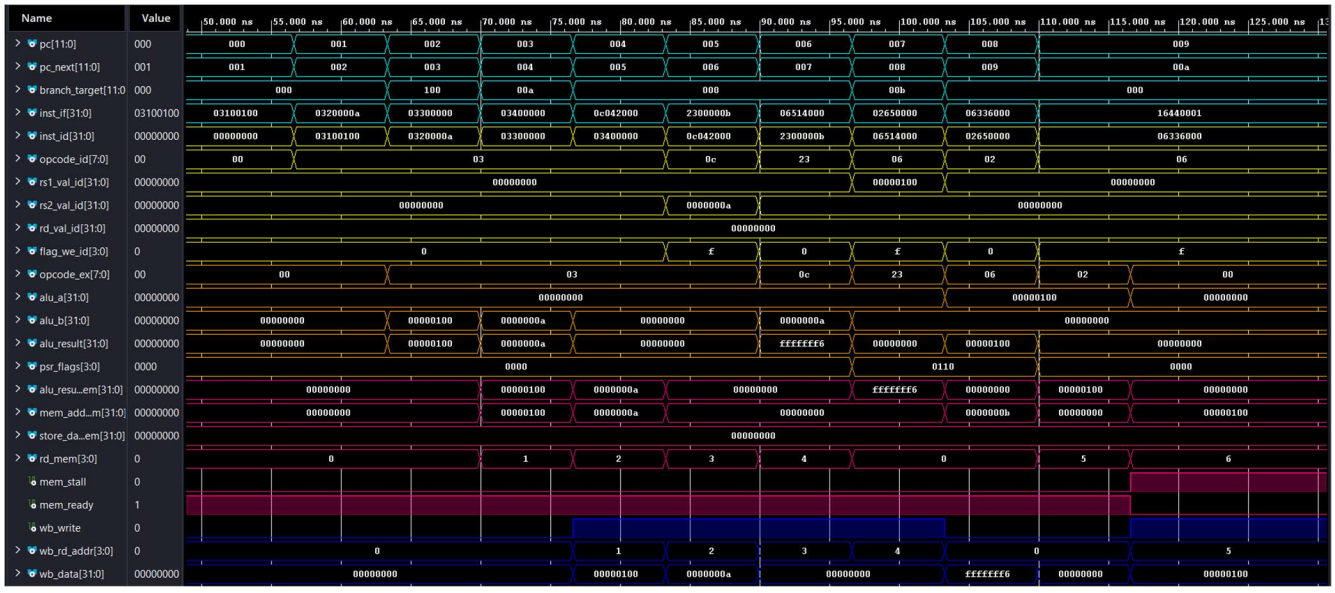


Figure 6. Detailed CPU pipeline waveform showing instruction progression, memory stall/ready, and writeback signals.